

Writing docs

How pages, routes, frontmatter, headings, callouts, and sidebars work in Almanac.

Shravan Goswami / <https://shravangoswami.com>

Pages live in a top-level directory

Docs content sits in `docs/` at the project root, not under `src/`. `docsLoader()` globs it into the docs collection, and Almanac injects one catch-all route that renders every entry. Adding a page means adding a file:

- `docs/index.md` → `/docs/`
- `docs/guides/writing-docs.md` → `/docs/guides/writing-docs/`
- `docs/reference/configuration.md` → `/docs/reference/configuration/`

An index file serves its directory. `docs.path` moves the directory, and `docs.base` moves the URL prefix (set `base: ""` to serve docs at the site root).

Files and directories starting with an underscore are skipped, so `docs/_partials/note.md` is not a page. `.md`, `.mdx`, and `.mdoc` are all globbed, but MDX and Markdoc only work if you also add the matching Astro integration.

Frontmatter

`title` is the only required field:

```
---
title: Writing docs
description: Shown under the title and used for meta tags and search.
slug: custom/url           # overrides the URL derived from the file path
sidebar:
  label: Writing           # shorter label for the sidebar
  order: 2                 # lower sorts first
  badge: New               # or { text: "New", variant: "tip" }
  hidden: false           # keep the page but drop it from the sidebar
draft: false              # true keeps the page out of the build
---
```

`title` and `description` show up in the page header, the browser tab title, the JSON-LD breadcrumb data, and the Pagefind index, so keep both short and accurate.

The schema also accepts these, and all of them are honoured:

Field	Effect
<code>template</code>	<code>"splash"</code> drops the sidebar, table of contents, pager, and the automatic title header, so the page composes its own hero
<code>tableOfContents</code>	<code>false</code> hides it; <code>{ minDepth, maxDepth }</code> overrides the site-wide depths for this page
<code>prev / next</code>	<code>false</code> removes that pager link; <code>{ link, label }</code> points it somewhere of your choosing
<code>editUrl</code>	<code>false</code> opts the page out of the site-wide edit link; a string replaces it
<code>lastUpdated</code>	A date overrides the git commit date; <code>false</code> hides the line
<code>head</code>	Extra tags in this page's <code><head></code> , merged after the site-wide ones

For example, a landing page with no chrome and a pager that only goes forward:

```

---
title: Welcome
template: splash
prev: false
next: { link: /docs/start/installation/, label: Install it }
---

```

Headings populate the table of contents

The table of contents on the right of every docs page is generated from the page’s `##` and `###` headings, in document order. You never maintain it by hand. As you scroll, an `IntersectionObserver` highlights the heading you’re in, and on narrow screens the same list collapses into an “On this page” panel.

Deeper headings (`####` and below) are rendered in the page but left out of the table of contents.

Linking between docs pages

Link with a path relative to the current page, not a root-relative one:

See `[Theming](../guides/theming/)` for the color system.

Almanac sites are often served under a base path (this one is served under `/almanac`, see [Deployment](#)). A relative link resolves correctly under any base; a root-relative link like `/docs/guides/theming/` skips the base path and 404s in production.

Callouts

Wrap a paragraph in a `<div>` with class `callout note` or `callout warning` to get a styled admonition. The styles ship with the framework, so there is nothing to import:

```

<div class="callout note">
Notes are for helpful context that isn't essential to following the instructions.
</div>

```

```

<div class="callout warning">
Warnings are for things that break a build or lose data if skipped.
</div>

```

Sidebars

Leave `docs.sidebar` out of your config and Almanac autogenerates the navigation: pages are grouped by their top-level directory, root-level pages lead under an “Overview” heading, and each group is sorted by `sidebar.order` first and alphabetically by label after. New files appear on their own.

Set `docs.sidebar` and you control the order instead. An entry can be a doc id, an explicit doc with a new label, an external link, a nested category, or an autogenerated directory:

```

sidebar: [
  "index",
  { doc: "start/installation", label: "Install" },
  { link: "https://astro.build", label: "Astro" },
  { label: "Guides", items: ["guides/writing-docs", { label: "Deep", items: ["guides/search"] }] },
  { autogenerated: { directory: "reference", collapsed: true } },
]

```

Ids that don’t resolve to a page are dropped rather than failing the build, which means a typo shows up as a missing sidebar link. With an explicit sidebar, adding a page does not add it to the navigation: add its id here too.

The previous/next pager at the foot of each page walks the flattened sidebar, so sidebar order is also reading order. External links are excluded from it.